

Neural Networks: Perceptrons

Video Lesson

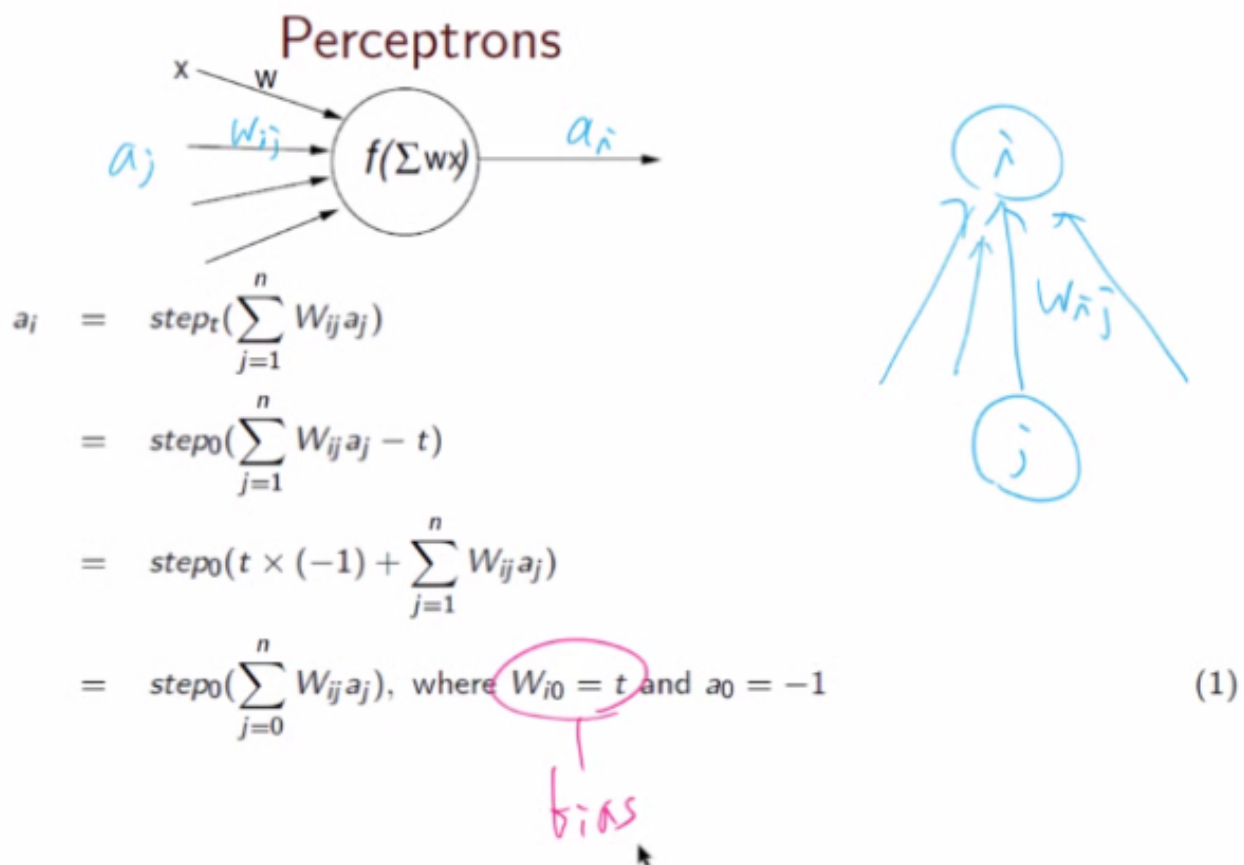
Duration: 00:17:34

Slide Deck

M12S04 (<https://canvas.tamu.edu/courses/430127/files/92570697?wrap=1>) 

Lesson Transcript

In this lecture, we will look into perceptrons, the simplest form of neural network.



Here is a simple perceptron unit. It takes in inputs and computes the input times the weight on each connection, sums them up, and passes it through the activation function, which is a step function with a threshold of t , and as a result, it generates the output a_i .

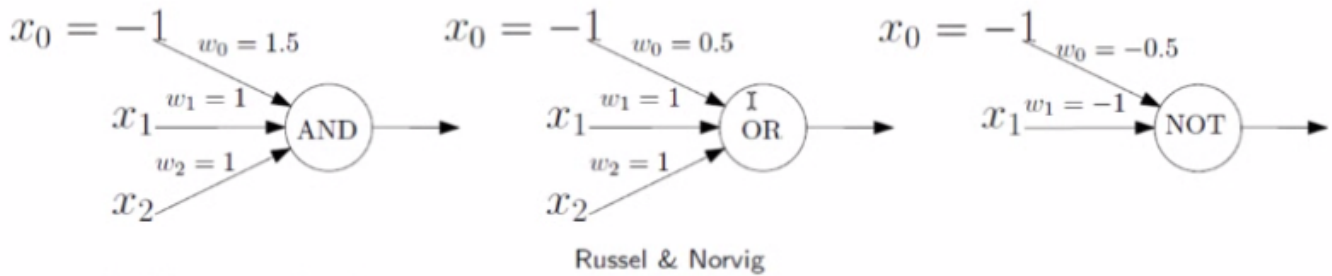
As we can see, the output neuron is indexed as i , and the input neurons are indexed as j . And by convention in the neural networks literature, when we have this kind of connection, we use W_{ij} to indicate this connection going from j to i . The reason is that this connection belongs to the output side.

Using the same trick that we saw in the previous lecture, we can turn this threshold t and incorporate that into the input.

So, supposing this was x , then t is now subtracted from that x , and we set the threshold to 0. And as we have seen earlier, these two are equivalent. Next, we do some manipulation here, turning $-t$ into $t \times -1$. And if we treat t as W_{i0} and -1 as a_0 , the input, then this whole thing now reduces to a single summation with j now ranging from 0 to n instead of j ranging from 1 to n . With this, we effectively turn this threshold t into a learnable parameter where the t was, again, W_{i0} .

In the neural network literature, this particular term that is adjustable is called the bias. So, we should not confuse this bias with the bias in inductive bias that we saw earlier in the previous

Boolean Logic Gates with Perceptron Units

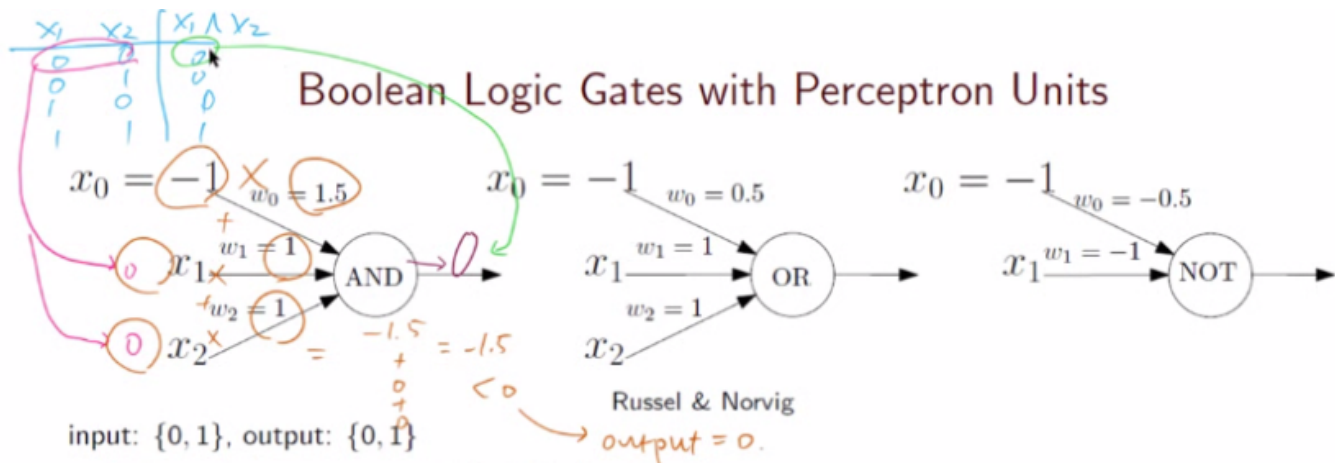


input: $\{0, 1\}$, output: $\{0, 1\}$

- Perceptrons can represent basic boolean functions.
- Thus, a network of perceptron units can compute any Boolean function.

What about XOR or EQUIV?

What can we do with this perceptron unit? Here we have some examples that show how to use this perceptron with specific weight assignments to perform the AND, OR, and NOT functions. As we know, if we have these three logic gates, we can implement any computer.



input: $\{0, 1\}$, output: $\{0, 1\}$

- Perceptrons can represent basic boolean functions.
- Thus, a network of perceptron units can compute any Boolean function.

What about XOR or EQUIV?

Let us consider the AND function. So, here is the truth table: 0, 0, 0; 0, 1, 0; 1, 0, 0; and finally, 1, 1, 1.

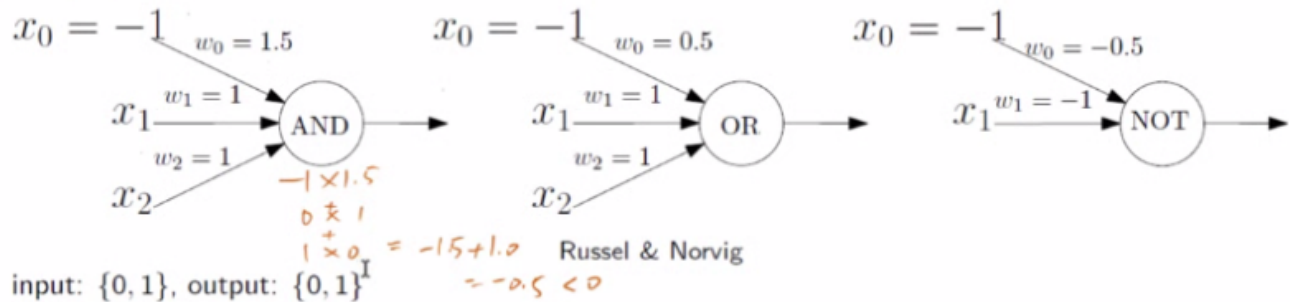
For each sample in the dataset, let us take the input values 0 and 0 for x_1 and x_2 and plug them into this perceptron unit. Then we want to compute the output value and compare that to the truth

value indicated in the dataset.

To compute output, we first multiply the bias input value -1 times the bias rate of 1.5 to get -1.5. We add that to $0 \times 1 + 0 \times 1$. So, the weighted sum becomes -1.5, and since that is less than 0, the output would also be 0, as shown here. That is exactly the same as the target value of 0.

x_1	x_2	$x_1 \wedge x_2$
0	0	0
0	1	0
1	0	0
1	1	1

Boolean Logic Gates with Perceptron Units



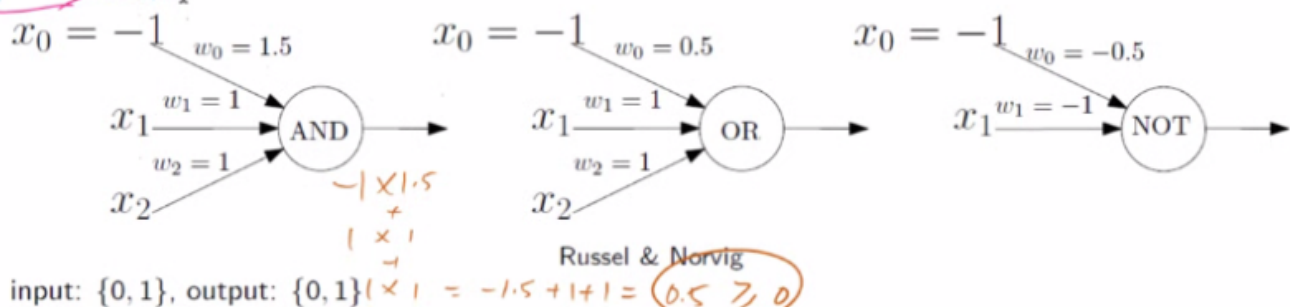
- Perceptrons can represent basic boolean functions.
- Thus, a network of perceptron units can compute any Boolean function.

What about XOR or EQUIV?

Let us try the next input. $x_1 = 0$ and $x_2 = 1$. So, we plug in these values here. Then what is computed here would be $-1 \times 1.5 + 0 \times 1 + 1 \times 1$. So, we have $-1.5 + 0 + 1$, and as a result, we get -0.5, which is less than 0. So again, the output would be 0, which is the correct outcome.

x_1	x_2	$x_1 \wedge x_2$
0	0	0
0	1	0
1	0	0
1	1	1

Boolean Logic Gates with Perceptron Units



- Perceptrons can represent basic boolean functions.
- Thus, a network of perceptron units can compute any Boolean function.

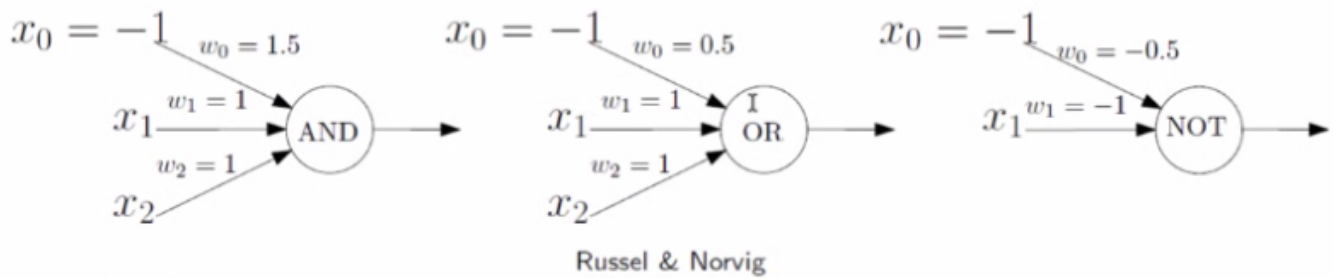
What about XOR or EQUIV?

When we repeat that with the third input, we get the same result because it is symmetric. Let us

look at the last input. When $x_1 = 1$, $x_2 = 1$, would we get 1 as the output?

So, if we plug in $x_1, 1, x_2, 1$, then this weighted sum would be $-1 \times 1.5 + 1 \times 1 + 1 \times 1$. So, that is $-1.5 + 1 + 1 = 0.5$. Because that is greater than or equal to 0, the output would be 1, and that matches with the target value in the dataset.

Boolean Logic Gates with Perceptron Units



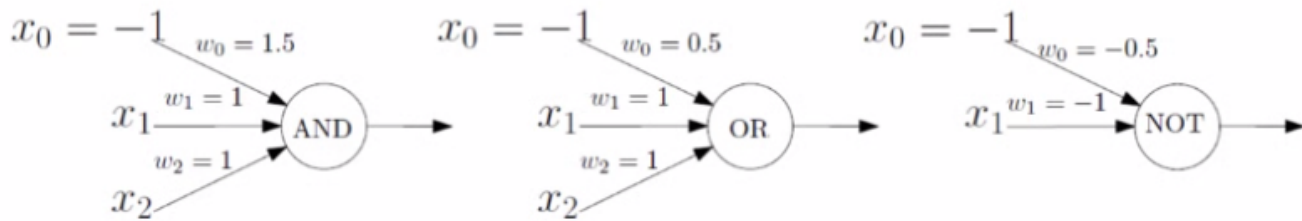
input: $\{0, 1\}$, output: $\{0, 1\}$

- Perceptrons can represent basic boolean functions.
- Thus, a network of perceptron units can compute any Boolean function.

What about XOR or EQUIV?

We can try this exercise for the rest of these examples OR and NOT, and we will find that these weight values would give us the correct answer. As a result of this, if we link up these perceptron units in an arbitrary manner, we can compute any boolean function, and hence we can implement any computer.

Boolean Logic Gates with Perceptron Units



Russel & Norvig

input: $\{0, 1\}$, output: $\{0, 1\}$

- Perceptrons can represent basic boolean functions.
- Thus, a network of perceptron units can compute any Boolean function.

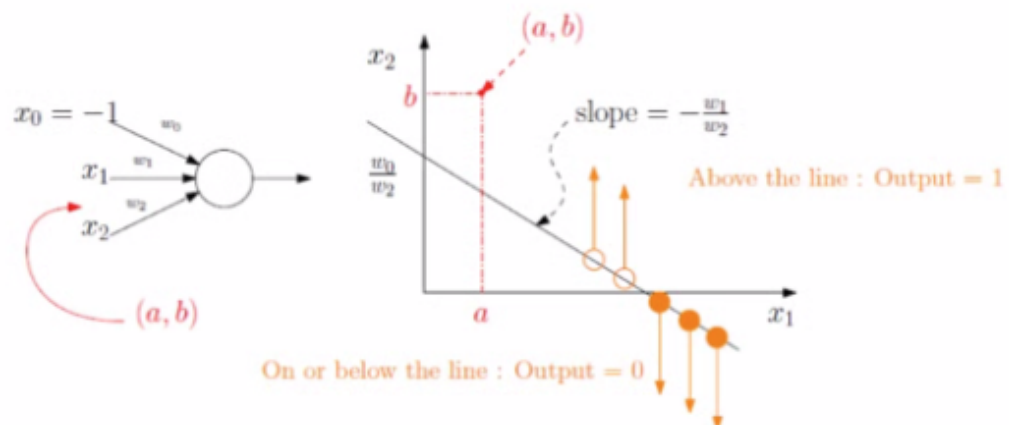
What about XOR or EQUIV?

x_1	x_2	$x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

We can also consider whether a single perceptron unit like that, if we set the connection rates appropriately, would be able to learn this kind of function, the XOR function, or equivalence, which is the negation of the XOR function.

The short answer is no, and we will see why that is the case and why that becomes a serious limitation of the perceptron.

What Perceptrons Can Represent



Perceptrons can only represent **linearly separable** functions.

- Output of the perceptron:

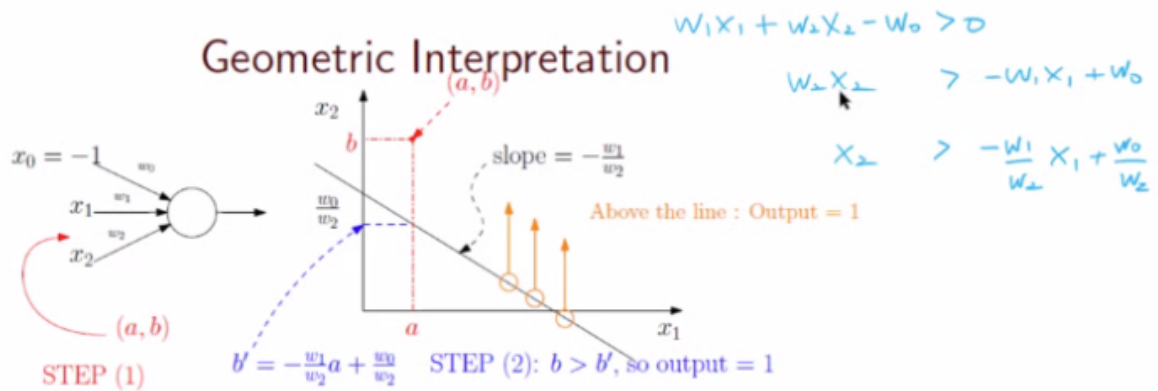
$$w_1 \times x_1 + w_2 \times x_2 - w_0 > 0, \text{ then output is } 1$$

$$w_1 \times x_1 + w_2 \times x_2 - w_0 \leq 0, \text{ then output is } 0$$

To understand what the perceptron does and understand its limitations, we need to know what the perceptron unit represents in a geometric sense. A quick summary is that perceptrons implement some kind of linear boundary. So, if we give inputs a and b , if that input occurs on that side of the boundary or the other side of the boundary, we get either 1 or 0 or the other way around.

We can look at what this perceptron does by first writing down the activation rule. It is simply the weighted sum of these and comparing that to 0. If it is greater than 0, then the output is 1. Otherwise, the output is 0.

Given this activation equation or inequality, when we plug in specific values a and b to x_1 and x_2 , whether this condition is met, whether that condition is met would be determined dependent on the position of this input a, b , relative to the decision boundary defined by these weight values.



- Rearranging $w_1 \times x_1 + w_2 \times x_2 - w_0 > 0$, then output is 1. With $(x_1, x_2) = (a, b)$, we get (if $w_2 > 0$)

$$\underbrace{x_2}_{=b} > \frac{-w_1}{w_2} \times \underbrace{x_1}_{=a} + \frac{w_0}{w_2} = b', \text{ that is, } b > b',$$

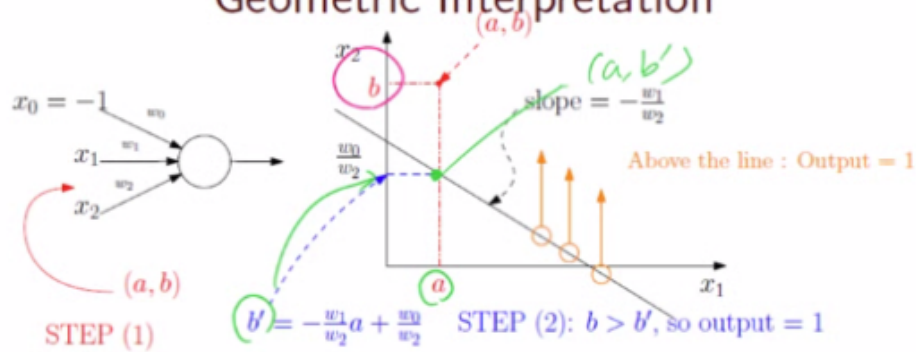
where points above the line, the output is 1, and 0 for those below the line (if $b < b'$). Consider:

$$y = \frac{-w_1}{w_2} \times x + \frac{w_0}{w_2}.$$

We can rearrange this activation rule into x_2 greater than $-w_1 / w_2 \times x_1 + w_0 / w_2$, and the steps that we need to follow are here. $w_1 x_1 + w_2 x_2 - 0 > 0$. So, we move these over to the other side, and we get $w_2 x_2 > -w_1 x_1 + w_0$.

Then we divide both sides with w_2 , and w_2 cancels out, so we get x_2 . On the right-hand side, we get this divided by w_2 and that divided by w_2 . This inequality maintains its direction only when $w_2 > 0$ because we are dividing both sides.

Geometric Interpretation



- Rearranging $w_1 \times x_1 + w_2 \times x_2 - w_0 > 0$, then output is 1. With $(x_1, x_2) = (a, b)$, we get (if $w_2 > 0$)

$$\underbrace{x_2}_{=b} > \underbrace{\frac{-w_1}{w_2} \times x_1}_{=a} + \frac{w_0}{w_2} = \underbrace{b'}_{}, \text{ that is, } b > b',$$

where points above the line, the output is 1, and 0 for those below the line (if $b < b'$). Consider:

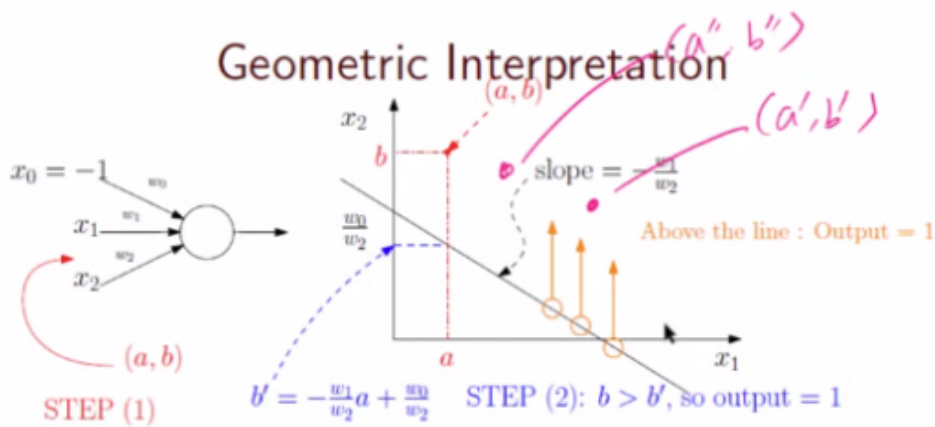
$$y = \frac{-w_1}{w_2} \times x + \frac{w_0}{w_2}.$$

Now let us plug in the values a and b as shown here. If the coordinate location for a and b is here, then when we plug in a to this right-hand side of the inequality where $x_1 = a$, then this value would be b' , which happens to be the location on that line defined by this slope of $-w_1 / w_2$. Then the y -intercept here of w_0 / w_2 .

For this particular example, then this b value is greater than this b' value, which is the height of this point on that line. This condition will be met.

As a result, the output would be 1.

Geometric Interpretation



- Rearranging $w_1 \times x_1 + w_2 \times x_2 - w_0 > 0$, then output is 1. With $(x_1, x_2) = (a, b)$, we get (if $w_2 > 0$)

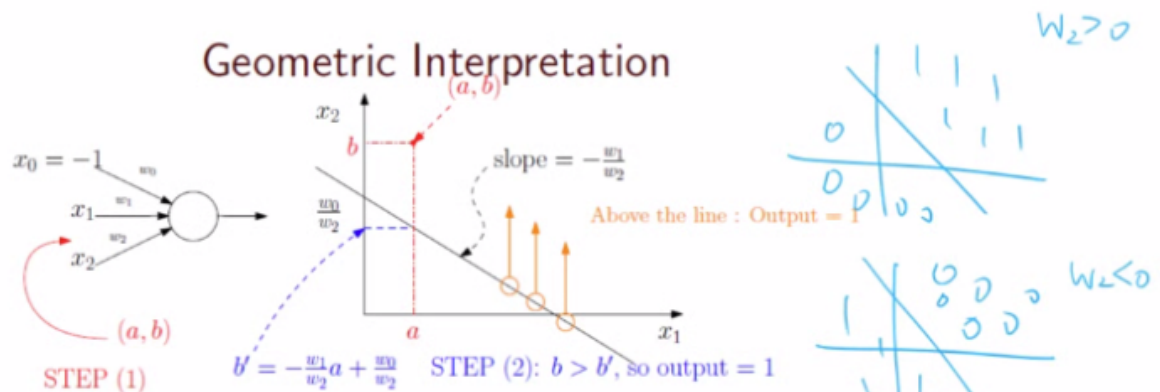
$$\underbrace{x_2}_{=b} > \frac{-w_1}{w_2} \times \underbrace{x_1}_{=a} + \frac{w_0}{w_2} = b', \text{ that is, } b > b',$$

where points above the line, the output is 1, and 0 for those below the line (if $b < b'$). Consider:

$$y = \frac{-w_1}{w_2} \times x + \frac{w_0}{w_2}.$$

Using the same logic, we can easily see that any input, a', b', a'', b' , that occurs above this line defined by these weight values would give the output of 1. On the other hand, if we have input on or below that line, the output would be 0.

Geometric Interpretation



- Rearranging $w_1 \times x_1 + w_2 \times x_2 - w_0 > 0$, then output is 1. With $(x_1, x_2) = (a, b)$, we get (if $w_2 > 0$)

$$\underbrace{x_2}_{=b} > \frac{-w_1}{w_2} \times \underbrace{x_1}_{=a} + \frac{w_0}{w_2} = b', \text{ that is, } b > b',$$

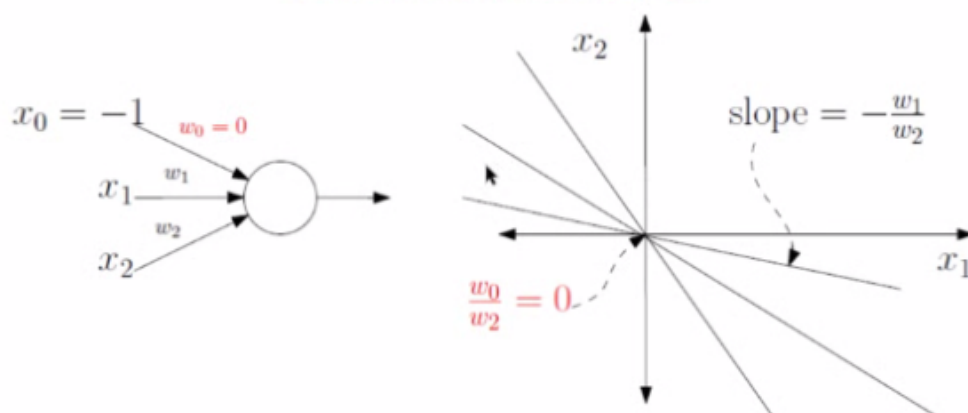
where points above the line, the output is 1, and 0 for those below the line (if $b < b'$). Consider:

$$y = \frac{-w_1}{w_2} \times x + \frac{w_0}{w_2}.$$

And here is when the sign of w_2 comes into play. Because we assume that it is greater than 0, this

is the result that we got. But when it is negative, then this inequality will flip the direction, and this is what we will get. If we are on or above this line, the output would be 0. If we are below this line, the output would be 1.

The Role of the Bias

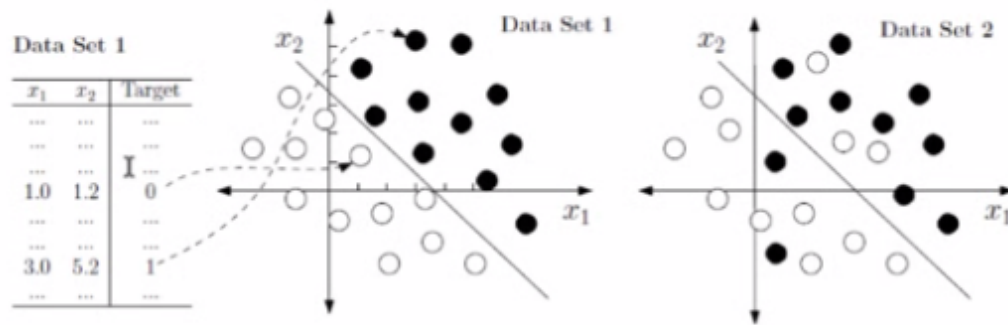


- Without the bias ($w_0 = 0$), learning is limited to adjustment of the slope of the separating line passing through the origin.
- Three example lines with different weights are shown.

Let us quickly look at the role of the bias, this bias rate. So, the bias input is always fixed to -1. But this bias weight value can be learned along with the other rates. If this weight value was 0, then it would effectively make this y-intercept to be 0. So, the only thing that we can change by adjusting the weights is to change the slope.

On top of the input data being separable by a straight line, it has to be separable by a straight line that passes through the origin.

Linear Separability



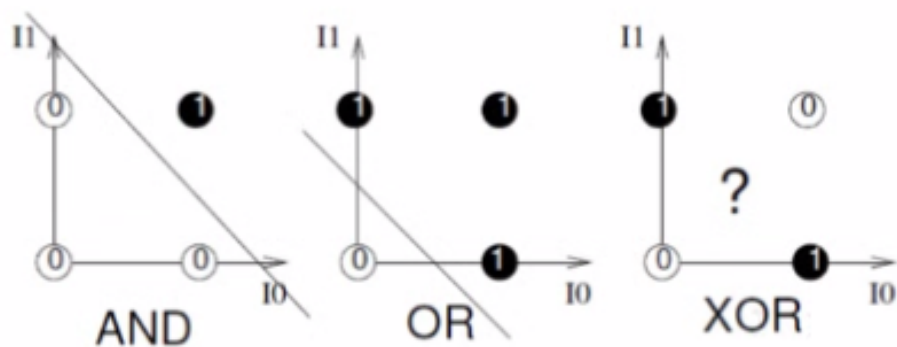
- Functions that take integer/real values output either 0 or 1.
- Data set 1: linearly separable (can draw a straight line between the classes).
- Data set 2: not linearly separable (perceptrons cannot represent such a function)

This brings us to the concept of linear separability. Let us consider these two datasets, Data Set 1 and Data Set 2. And here, I am showing Data Set 1 with the exact values. Given this dataset, we can plot them in the input space x_1 and x_2 . For example, this data point here has an x_1 value of 1, an x_2 value of 1.2, and this has a target value of 0.

On the other hand, this input here, corresponding to this sample in the data set, has an x_1 value of 3, and then an x_2 value of 5.2, and the target value is 1. So, 1 is represented as black, and 0 as white. If we plot all of these data points, and they look like this, then what we see is that we can draw a straight line between these two groups.

Data sets that have this kind of property are called linearly separable. Imagine another data set here, Data Set 2, and if we plot all of these x_1 , x_2 values and their corresponding target, and as a result, we get this, then we can clearly see that there is no way to draw a straight line between the black group and the white group. So, Data Set 1 is linearly separable, but Data Set 2 is not linearly separable, and we can train the perceptron to learn this dataset without error. Whereas for this kind of dataset that is not linearly separable, perceptrons cannot really learn to classify correctly.

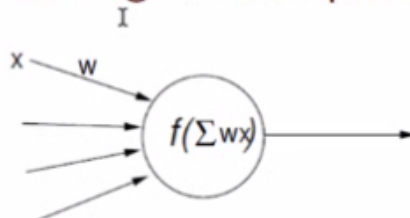
Linear Separability (cont'd)



- Perceptrons cannot represent XOR!
- Minsky and Papert (1969)

Here is another example of linearly separable and not linearly separable input. So, this is the dataset for the logical function AND, this is for OR, and then this is for XOR. As we can see in these two cases, we can draw a straight line between the black and white group. Whereas here, it is impossible to draw a straight line to separate the black ones and the white ones.

Learning in Perceptrons

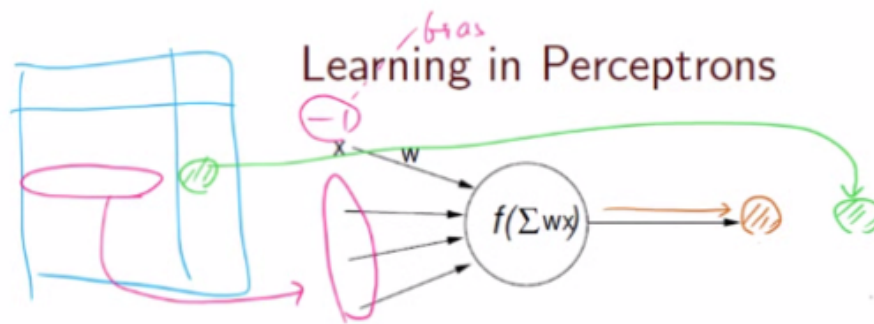


- The weights do not have to be calculated manually.
- We can train the network with (input,output) pair according to the following weight update rule:

$$W_j \leftarrow W_j + \alpha \times I_j \times Err,$$

where α is the learning rate parameter, I_j is the input, and $Err = DesiredOutput - NetworkOutput$.

So far, we looked at these perceptron units with specific rate values, but the real usage of perceptrons is when we have a dataset, and we want to learn these weight values automatically. To do that, what we have is the perceptron learning rule.



- The weights do not have to be calculated manually.
- We can train the network with (input,output) pair according to the following weight update rule:

$$W_j \leftarrow W_j + \alpha \times I_j \times Err,$$

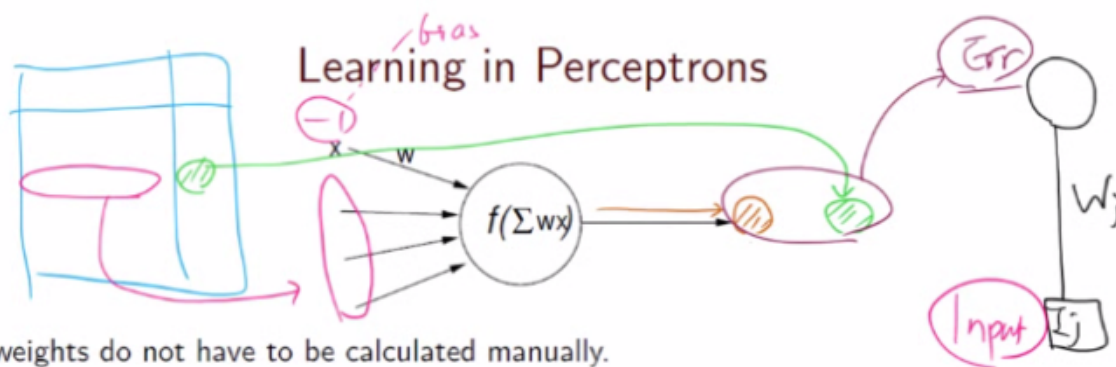
where α is the learning rate parameter, I_j is the input, and
 $Err = \text{DesiredOutput} - \text{NetworkOutput}.$



The weight update formula is $W_j = W_j$, the previous value, $+ \alpha$, which is a fixed learning rate constant, typically between 0 and 1. So, the value could be something like 0.1 or 0.01. And then input on that particular connection W_j . So, that is I_j times the error that is made.

How do we compute this error? Take particular input from the dataset, plug it into the perceptron, and let us say the first input is -1, the bias unit. And we compute the resulting output, either 1 or 0. So, that is the network output, and we take the target value from the dataset. That is the green value here. That is the desired output.

The error is basically the desired output minus the network output.



- The weights do not have to be calculated manually.
- We can train the network with (input,output) pair according to the following weight update rule:

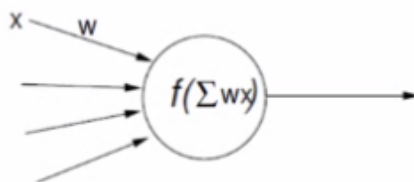
$$W_j \leftarrow W_j + \alpha \times I_j \times Err.$$

where α is the learning rate parameter, I_j is the input, and $Err = DesiredOutput - NetworkOutput$.



To illustrate, here is the input value, and here is the output of the perceptron. The weight W_j is updated proportionally to the input value times the error.

Learning in Perceptrons



- The weights do not have to be calculated manually.
- We can train the network with (input,output) pair according to the following weight update rule:

$$W_j \leftarrow W_j + \alpha \times I_j \times Err,$$

where α is the learning rate parameter, I_j is the input, and $Err = DesiredOutput - NetworkOutput$.

$$\begin{aligned} Err &= 0 - 1 = -1 \\ Err &= 1 - 0 = 1 \\ Err &= 0 - 0 = 0 \end{aligned}$$

When there is no error, or when the input value is 0, then this whole thing will be 0, so there will be no change in the weight value.

For example, here, if the desired output is 0 and the network output is 0, then there will be 0.

When the desired output is 1 and the network output is 1, again, the error will be 0.

One interesting thing here is that this weight value can be either increased or decreased depending on the type of error and the input value. If this input is always positive, then this error will determine the sign and hence whether W_j will be increased or decreased. When the desired output is 0, and the network output is 1, then the error would have a negative sign so that the weight value will be decreased.

On the other hand, if the desired output is 1, but the network output is 0, then the error will be positive 1, and as a result, the weight will be increased.

Exercise: Implementing the Perceptron

- It is fairly easy to implement a perceptron.
- You can implement it in any programming language: C/C++, Python, etc.

The perceptron is a very simple algorithm, so we can quickly implement it using C, C++, or Python, and this will be part of the homework programming assignment.

In the following lecture, we will look into multilayer perceptrons or back-propagation networks.